

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

The following parameter counts are the values obtained from the models implemented and trained in this experiment.

Architecture	Parameters	Main Contribution
LeNet-5	83,126	First CNN
AlexNet	7,508,810	ReLU + Dropout
VGG16	14,848,586	Deep 3×3 filters
GoogleNet	908,906	Inception Modules
ResNet50	24,114,826	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.

- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

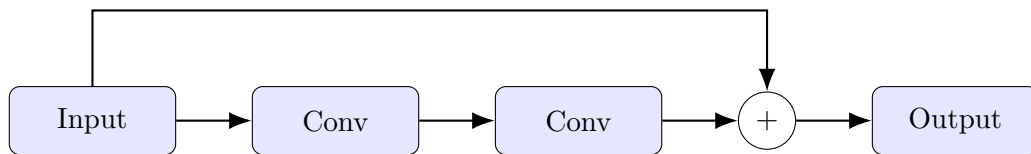


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

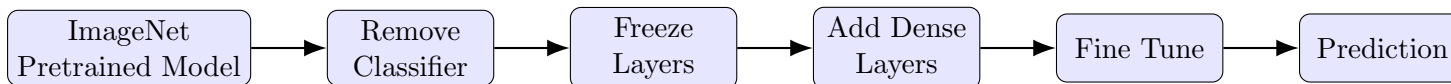


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots

The following plots were obtained from the experiment.

1. Sample CIFAR-10 images
2. Training and Validation Accuracy
3. Training and Validation Loss
4. Confusion Matrix
5. Misclassified Images (Optional)

Sample CIFAR-10 Images

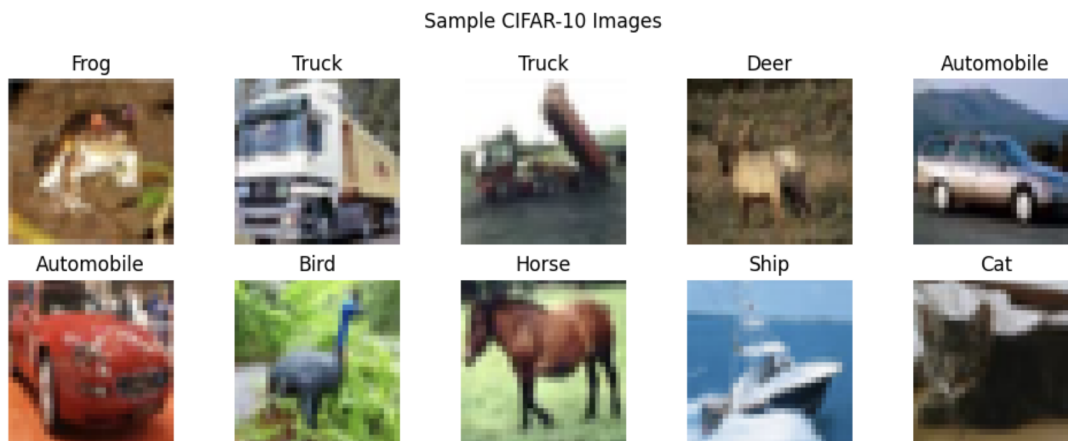


Figure 3: Sample CIFAR-10 Images

Inference: The images contain different classes such as animals, vehicles and objects, but many of them are very small and some objects are mixed with the background. Because the images are only 32×32 pixels, some classes can look similar, which can make classification difficult.

Training and Validation Accuracy

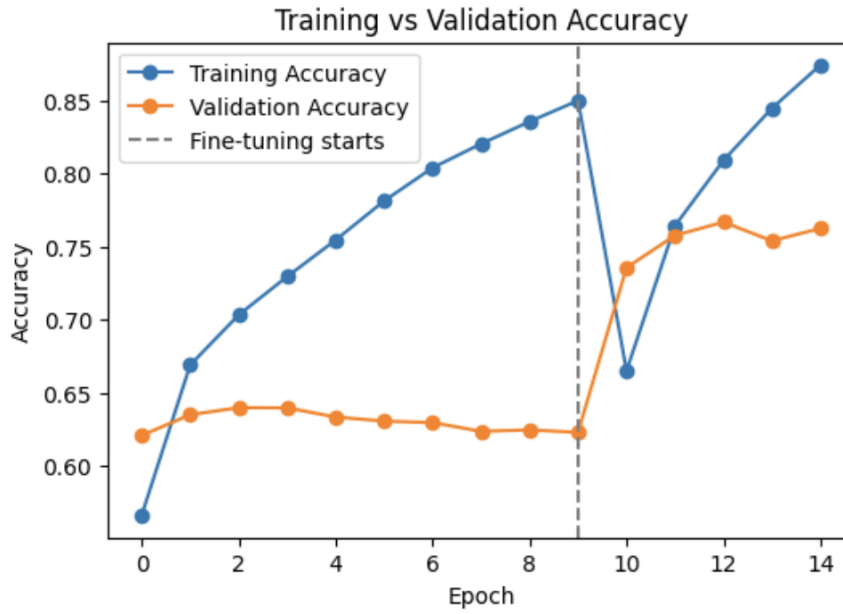


Figure 4: Training and Validation Accuracy

Inference: Training accuracy increases from about 57% to 85% during the frozen training stage as the classifier head learns from the pretrained features. At the fine-tuning boundary, training accuracy temporarily drops because block5 is now trainable and the model starts adjusting the pretrained features. It then recovers and reaches 87.45%, while validation accuracy improves from 62.29% to 76.26% after fine-tuning.

Training and Validation Loss

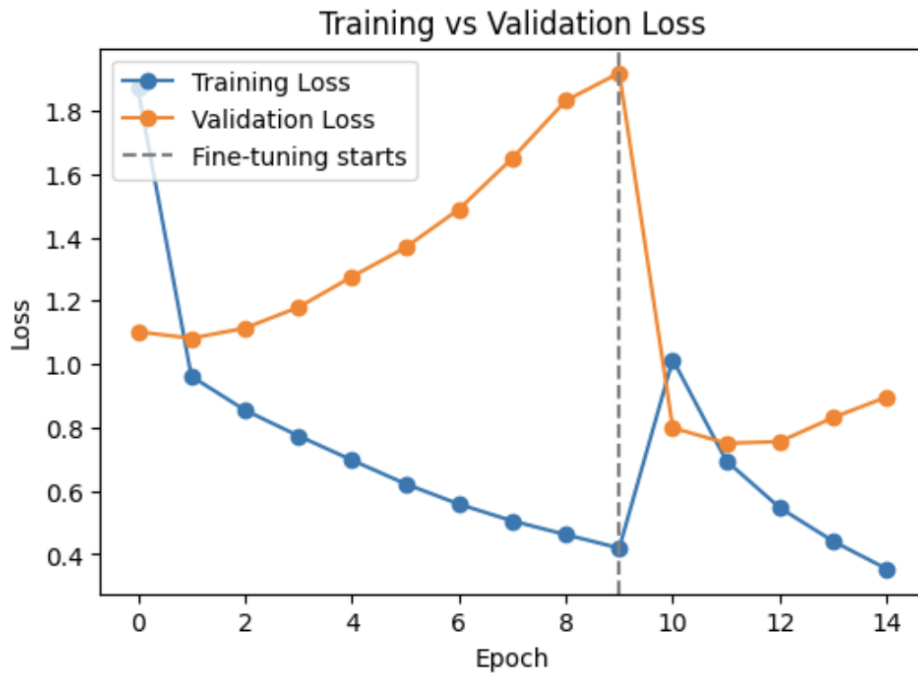


Figure 5: Training and Validation Loss

Inference: Training loss decreases from about 1.87 to 0.42 during the frozen training stage as the model becomes better at predicting the training images. At the start of fine-tuning, the loss temporarily changes because the last convolution block starts updating, and it then falls again to about 0.35. Validation loss first increases up to about 1.92 during frozen training, but after fine-tuning it drops to about 0.75, matching the improvement seen in validation accuracy.

Confusion Matrix

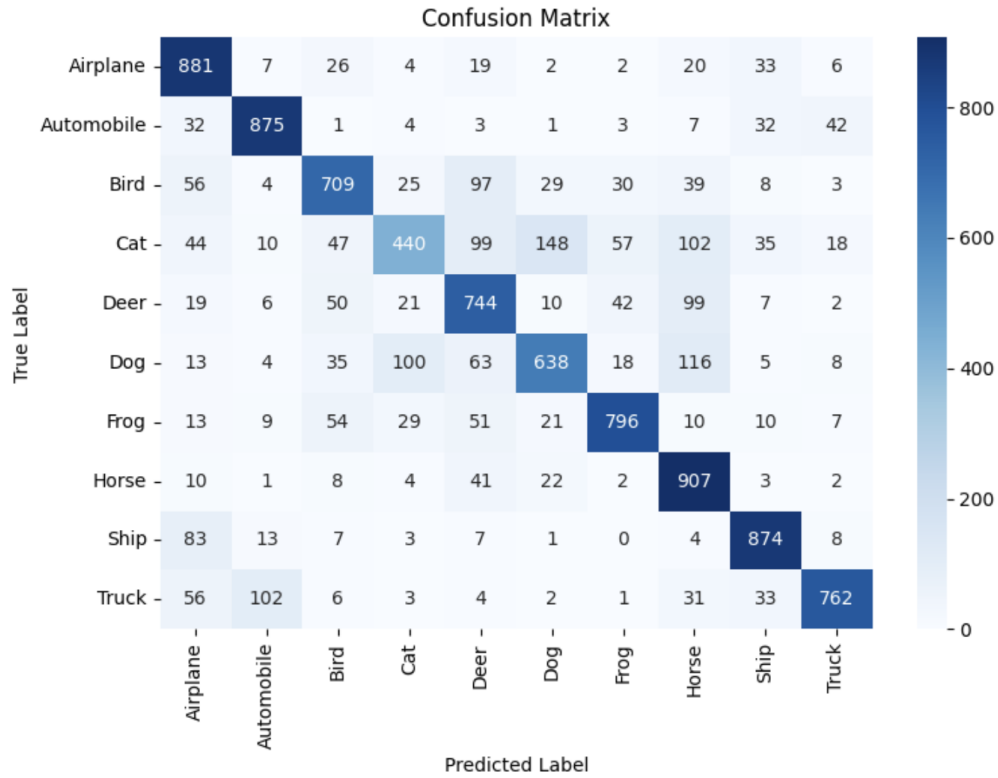


Figure 6: Confusion Matrix

Inference: The model performs best on Horse (907), Airplane (881), Automobile (875) and Ship (874), as these classes have the highest correctly classified images. Cat is the most difficult class, with only 440 images correctly classified, and it is mainly confused with Dog (148), Horse (102) and Deer (99). There is also noticeable confusion between Dog and Horse (116) and Truck and Automobile (102), showing that these visually similar classes are harder for the model to distinguish.

Misclassified Images (Optional)



Figure 7: Misclassified Images

Inference: The misclassified examples mainly occur between visually similar classes, such as Cat-Dog, Deer-Horse and Truck-Automobile. Some images also have a small object or a large amount of background, making the main object less clear. These examples are consistent with the larger errors seen in the confusion matrix.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy	87.45%
Testing Accuracy	76.26%
Precision	76.60%
Recall	76.26%
F1-score	75.79%
Training Time	275.69 seconds
Total Parameters	14,848,586

The final VGG16 model achieved 76.26% testing accuracy, with precision, recall and F1-score of 76.60%, 76.26% and 75.79% respectively. The training accuracy reached 87.45%, which is higher than the testing accuracy. Fine-tuning improved the validation accuracy from 62.29% to 76.26%.

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	60.24	1.33 min
AlexNet	7,508,810	72.43	3.82 min
VGG16	14,848,586	76.26	4.59 min
GoogleNet	908,906	73.07	4.50 min
ResNet50	24,114,826	65.81	2.51 min

Inference: VGG16 gives the highest accuracy at 76.26%, followed by GoogleNet at 73.07% and AlexNet at 72.43%. VGG16 has more parameters than GoogleNet, but GoogleNet still reaches 73.07%

with only 908,906 parameters. This shows that a larger number of parameters did not directly give better accuracy or lower training time in this experiment.

18.3 Frozen vs Fine-tuned VGG16

Setting	Validation Accuracy
Frozen	62.29%
Fine-tuned	76.26%

Inference: The frozen model gives 62.29% accuracy, while fine-tuning gives 76.26%, which is an improvement of about 14 percentage points. The last convolution block was allowed to change during fine-tuning, so the pretrained features could adapt better to the CIFAR-10 images.

18.4 Adam vs SGD

Optimizer	Final Validation Accuracy
Adam	64.71%
SGD	63.32%

Inference: Adam reaches 64.71% accuracy compared with 63.32% for SGD in this experiment. Adam also gives higher validation accuracy in the five epochs used here. With the learning rate and other settings used, Adam therefore gave slightly better results.

18.5 Hyperparameter Study Results

The following table shows the accuracy and training time obtained for the different hyperparameter settings tested with the VGG16 model.

Hyperparameter	Setting	Accuracy (%)	Training Time (s)
Learning Rate	0.001	63.129997	153.556588
Learning Rate	0.0001	63.380003	158.871442
Batch Size	16	62.830001	244.972745
Batch Size	32	63.870001	152.706186
Batch Size	64	63.010001	124.104708
Epochs	10	62.699997	153.858191
Epochs	20	61.009997	299.658215
Dense Units	128	63.200003	153.595589
Dense Units	256	62.410003	153.061870
Optimizer	Adam	62.910002	152.966071
Optimizer	SGD	63.489997	158.756235
Frozen Layers	All	63.160002	163.443840
Frozen Layers	Partial	10.000000	203.305080

Inference: Batch size 32 gave the highest accuracy among the tested batch sizes at 63.870001%, while batch size 64 had the lowest training time. Increasing the epochs from 10 to 20 increased the training time considerably, but the accuracy decreased from 62.699997% to 61.009997%. The tested dense units and learning rates produced only small accuracy differences.

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was a major breakthrough because it showed that a deep CNN can give very good image classification results using GPU-based training. It also used ReLU and Dropout. In our experiment, AlexNet achieved 72.43% accuracy, which was much higher than LeNet-5 at 60.24%.

2. Why does VGG16 use only 3×3 convolution filters?

VGG16 uses small 3×3 filters and stacks them in multiple layers to learn features step by step. This also allows the network to become deeper while keeping the convolution filters small. In our experiment, VGG16 achieved the highest accuracy of 76.26% among the five compared models.

3. Explain the advantages of the Inception module.

The Inception module uses different convolution sizes in parallel, so features at different scales can be captured from the same image. The GoogleNet model in our experiment achieved 73.07% accuracy with only 908,906 parameters, which is much fewer than VGG16 and AlexNet.

4. What is the purpose of residual learning?

Residual learning uses skip connections so that the network can learn a residual mapping instead of learning the complete mapping directly. This helps deep networks pass information through many layers. In our experiment, ResNet50 had 24,114,826 parameters but achieved 65.81% accuracy.

5. Differentiate LeNet and ResNet.

LeNet is a much smaller CNN, while ResNet50 is a much deeper network with residual connections. In our experiment, LeNet had 83,126 parameters and achieved 60.24% accuracy, while ResNet50 had 24,114,826 parameters and achieved 65.81%.

6. What is Transfer Learning?

Transfer learning uses a model that has already learned useful image features from a large dataset and adapts those features to a new dataset. In this experiment, VGG16 was loaded with pretrained ImageNet weights, its convolution layers were first frozen, and a new classifier was added for the 10 CIFAR-10 classes.

7. Why is fine tuning required?

Fine-tuning allows selected pretrained layers to change according to the new dataset. In our experiment, the frozen VGG16 gave 62.29% validation accuracy, while fine-tuning the last convolution block increased it to 76.26%. This shows that adapting the pretrained features helped the model perform better on CIFAR-10.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution increases the receptive field by adding gaps between kernel elements without directly increasing the number of kernel parameters. Transpose convolution is used to increase the spatial size of a feature map through learnable upsampling. Therefore, the main purpose of dilated convolution is to capture a larger area, while transpose convolution is used to increase output size.

9. Why do pretrained models converge faster?

A pretrained model already has useful image features instead of starting with completely random features. In our experiment, the VGG16 model reached about 62% validation accuracy during the frozen training stage and then improved further after fine-tuning. This shows that the pretrained features gave the model a useful starting point.

10. Compare the computational complexity of LeNet and ResNet.

LeNet is much smaller than ResNet50. In our results, LeNet had 83,126 parameters and took 1.33 minutes, while ResNet50 had 24,114,826 parameters and took 2.51 minutes. ResNet50 therefore used many more parameters, although the accuracy improvement over LeNet was only from 60.24% to 65.81% in this experiment.

20. Additional Exercises

1. Implement transfer learning using VGG16.

The VGG16 transfer-learning model was implemented using ImageNet pretrained weights, a frozen convolutional base, Global Average Pooling, a Dense layer and a 10-class Softmax output.

2. Repeat the experiment using ResNet50.

ResNet50 was also implemented using ImageNet pretrained weights. After fine-tuning the last convolution block, it achieved 65.81% final accuracy with 24,114,826 parameters and a recorded training time of 2.51 minutes.

3. Compare Adam and SGD optimizers.

Optimizer	Final Validation Accuracy
Adam	64.71%
SGD	63.32%

Adam achieved 64.71% final validation accuracy, while SGD achieved 63.32%. With the settings used in this experiment, Adam performed slightly better because its adaptive learning rate helped the model update the weights more effectively.

4. Compare training with frozen layers and fine tuning.

Setting	Validation Accuracy
Frozen	62.29%
Fine-tuned	76.26%

The frozen VGG16 model achieved 62.29% validation accuracy. After unfreezing the last convolution block and fine-tuning, the accuracy increased to 76.26%. This shows a clear improvement from adapting the pretrained features.

5. Evaluate the model using Precision, Recall and F1-score.

The final VGG16 model achieved 76.60% macro precision, 76.26% macro recall and 75.79% macro F1-score.

6. Compare the performance of LeNet, AlexNet and ResNet.

Among these three models, AlexNet achieved the highest accuracy at 72.43%, followed by ResNet50 at 65.81% and LeNet-5 at 60.24%. Their parameter counts were 7,508,810, 24,114,826 and 83,126 respectively.

21. Source Code

The complete source code and experiment notebook are available in the following GitHub repository:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_4

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>